Submission Worksheet

Submission Data

Course: IT202-450-M2025
Assignment: IT202 Milestone 1
Student: Colin R. (ctr26)
Status: Submitted | **Worksheet Progress:** 95%
Potential Grade: 9.67/10.00 (96.70%)
Received Grade: 0.00/10.00 (0.00%)
Started: 7/10/2025 9:06:32 PM
Updated: 7/10/2025 11:11:11 PM
Grading Link: https://learn.ethereallab.app/assignment/v3/IT202-450-M2025/it202-milestone-1/grading/ctr26
View Link: https://learn.ethereallab.app/assignment/v3/IT202-450-M2025/it202-milestone-1/view/ctr26

Instructions

- Overview Link: https://youtu.be/IDtqdaLwcVg

1. Refer to Milestone1 of this doc: https://docs.google.com/document/d/1XE96a8DQ52Vp49XACBDTNCq0xYDt3kF29cO88EWVwfo/view
2. Ensure you read all instructions and objectives before starting.
3. Ensure you've gone through each lesson related to this Milestone
4. Switch to the Milestone1 branch
    1. `git checkout Milestone1` (ensure proper starting branch)
    2. `git pull origin Milestone1` (ensure history is up to date)
5. Fill out the below worksheet
    - Ensure there's a comment with your UCID, date, and brief summary of the snippet in each screenshot
    - Ensure proper styling is applied to each page
    - Ensure there are no visible technical errors; only user-friendly messages are allowed
6. Once finished, click "Submit and Export"
7. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
    1. `git add .`
    2. `git commit -m "adding PDF"`
    3. `git push origin Milestone1`
    4. On Github merge the pull request from `Milestone1` to `dev`
    5. On Github create a pull request from `dev` to `prod` and immediately merge. (This will trigger the `prod` deploy to make the heroku prod links work)
8. Upload the same PDF to Canvas
9. Sync Local
10. `git checkout dev`
11. `git pull origin dev`

Section #1: (2 pts.) Feature: User Will Be Able To Register A New Account

Progress: 100%

Task #1 (0.67 pts.) - Validation

Progress: 100%

Details:

- Show the relevant code snippets for each validation layer
- Show the relevant demo of each validation layer
- Briefly explain how the validation steps work at each layer

Task #1 (0.33 pts.) - HTML Form/Validation

Progress: 100%

Details:

- System should allow the user to correct the error without wiping/clearing the form (for the PHP side this includes sticky-form logic to keep the username/email address entries)

Part 1:

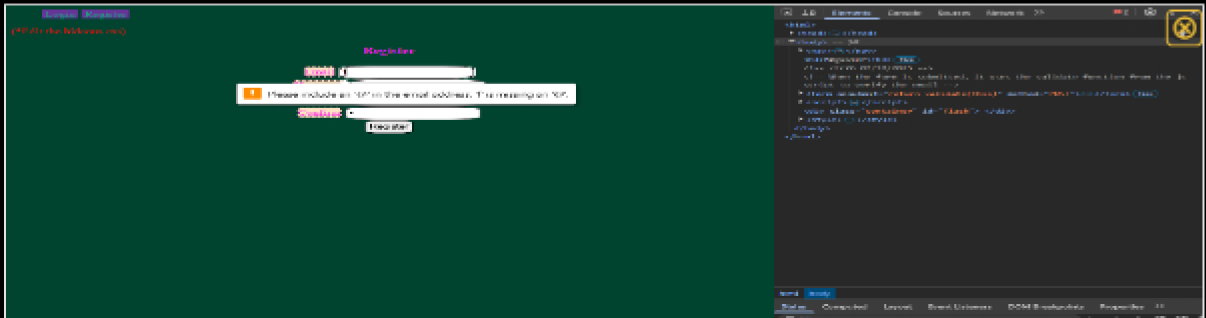
Progress: 100%

Details:

- Show the code related to the register page's form (HTML validation)
- Show examples of each validation message (you may be able to capture this in one or few screenshots)

```
<!   ctr26 07/10/2025   >
<!   when the form is submitted, it uses the validate function from the js script to verify the email
<form onsubmit="return validate(this)" method="POST">
    <div>
        <label for="email">Email</label>
        <input id="email" type="email" name="email" required />
    </div>
    <div>
        <label for="username">Username</label>
        <input type="text" name="username" required maxlength="30" />
    </div>
    <div>
        <label for="pw">Password</label>
        <input type="password" id="pw" name="password" required minlength="8" />
    </div>
    <div>
        <label for="confirm">Confirm</label>
        <input type="password" name="confirm" required minlength="8" />
    </div>
    <input type="submit" value="Register" />
</form>
```

HTML code/comment



HTML output

Saved: 7/10/2025 9:31:47 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The code validates the password by utilizing a function nested in the script tag. This allows the html to run validation steps using javascript commands.

Saved: 7/10/2025 9:31:47 PM

Task #2 (0.33 pts.) - JS Validation (validate() function)

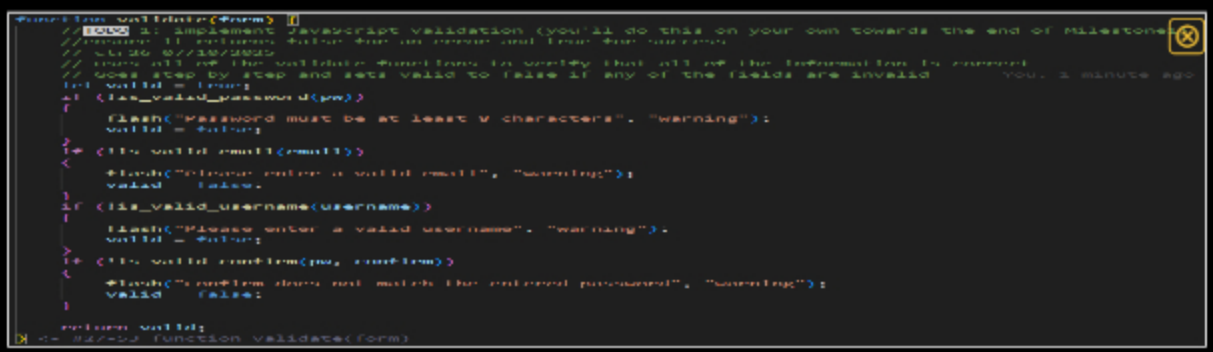
Progress: 100%

Part 1:

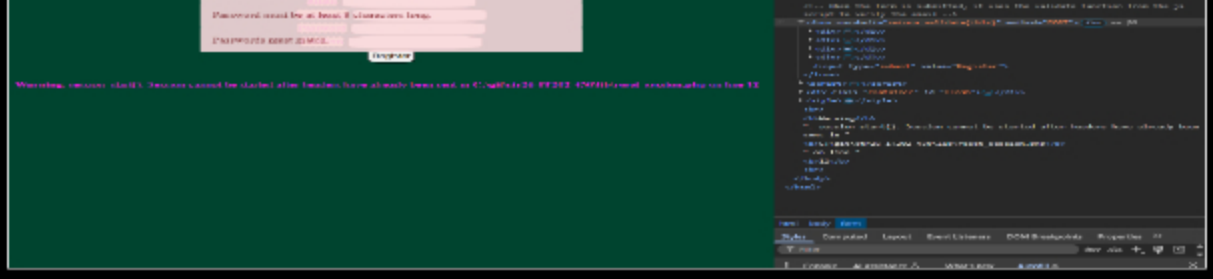
Progress: 100%

Details:

- Show the code related to the register page's validate() function
- Show examples of each validation message [username format, email format, password format, password doesn't match confirm](you may be able to capture this in one or few screenshots)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag



JS code/comment



JS output

Saved: 7/10/2025 9:33:06 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The validate function check each of the different requirements one by one and sets the valid boolean to false if any of them do not meet the requirements. This boolean is then output asthe final result.

Saved: 7/10/2025 9:33:06 PM

Task #3 (0.33 pts.) - PHP Validation (steps before the DB call)

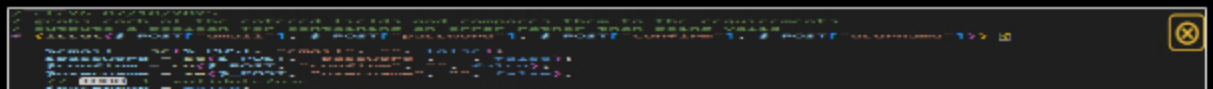
Progress: 100%

Part 1:

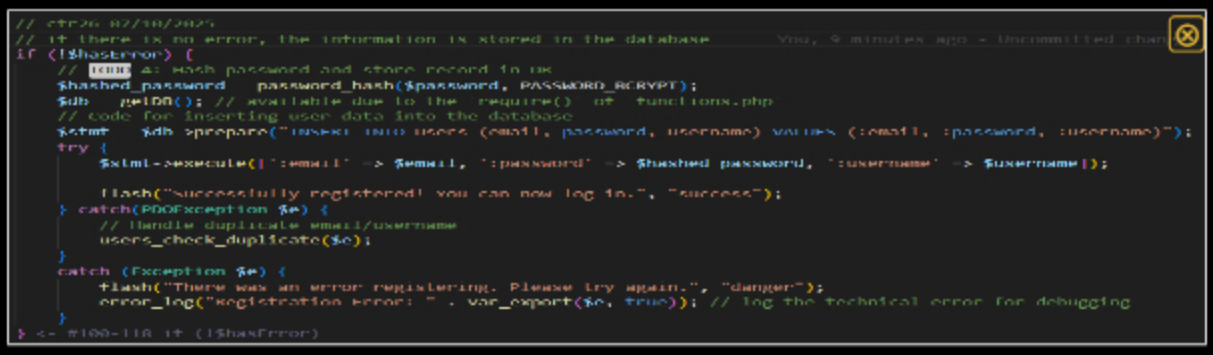
Progress: 100%

Details:

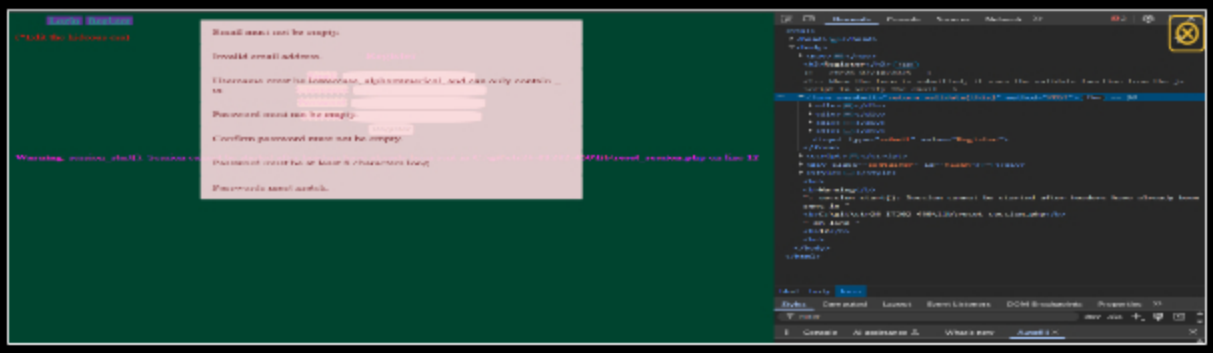
- Show the code related to the register page's PHP validation
- Show examples of each validation message [username format, email format, password format, invalid password, password doesn't match confirm, username taken, email taken] (you may be able to capture this in one or few screenshots)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag and temporarily override the validate() function or use the provided helper script in the instructions
- Include the outputs related to username/email already in use



PHP code/comment 1/2



PHP code/comment 2/2



PHP output

Saved: 7/10/2025 9:51:11 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The code goes through each of the requirements one by one and updates a boolean to true if there is an error. Once all requirements have been checked, if there is no error, the information is stored in the database. Note: I pasted the wrong thing into the url box and I am not able to remove it. The correct urls are still present.

Saved: 7/10/2025 9:51:11 PM

Task #2 (0.67 pts.) - Related URLs

Progress: 100%

Details:

- Include the direct link to this file from the Milestone branch (should end in .php)
- Include the heroku prod link to this file (Just grab the base prod url and manually enter the path to the file)
- Include the related pull request link for this feature

URL #1

https://learn.ethereallab.app/assignment/v3/IT202-450-M2025/%3C?php%20require(__DIR__%20.%20%22/%3E%20%3Ch3%3ERegister%3C/h3%3E%20%3Cform%20onsubmit=%22return%20validate(this)%22%20me
php%20//TODO%202:%20add%20PHP%20Code%20if%20(isset($_POST[%22email%22]%20$_POST[%22pa
%22,%20%22danger%22);%20%20%20%20%20%20%20%20%20$hasError%20=%20true;%20%20%20%20%2
%3Eprepare(%22INSERT%20INTO%20Users%20(email,%20password,%20username)%20VALUES%20(:emai
%3Eexecute([%27:email%27%20=%3E%20$email,%20%27:password%27%20=%3E%20$hashed_password,%
%3E%20%3C?php%20require(__DIR__%20.%20%22/../../partials/flash.php%22);%20reset_session();%20?%3

URL #2

https://github.com/ColinRafferty7/ctr26-IT202-450/Milestone1/public_html/project/register.php

URL: https://github.com/ColinRaff

URL #3

https://ctr26-it202-450-prod-2ae7f60cad14.herokuapp.com/project/register.php

URL: https://ctr26-it202-450-prod-2

URL #4

https://github.com/ColinRafferty7/ctr26-IT202-450/pull/18

URL: https://github.com/ColinRaff

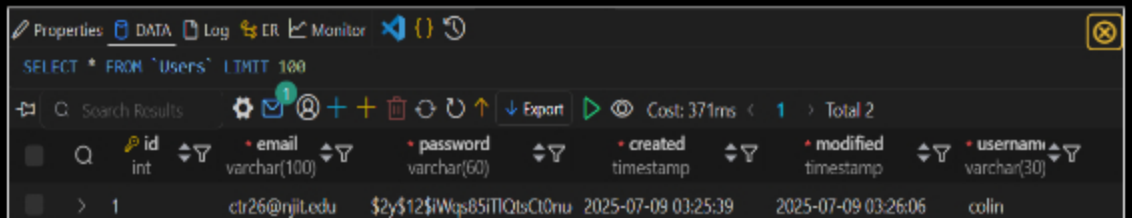
Saved: 7/10/2025 9:49:57 PM

Task #3 (0.67 pts.) - Database - Users table

Progress: 100%

Details:

- Add a screenshot of the database view from vs code showing a valid registered user (preferrably a recent one during evidence capturing)



3	test@test.com	$2y$12$cTKP7zoYaQlmTskY	2025-07-10 02:08:57	2025-07-10 02:08:57	test1

Users table

Saved: 7/10/2025 9:52:55 PM

Section #2: (2 pts.) Feature: User Will Be Able To Login To Their Account

Progress: 100%

Task #1 (0.67 pts.) - Validation

Progress: 100%

Details:

- Show the relevant code snippets for each validation layer
- Show the relevant demo of each validation layer
- Briefly explain how the validation steps work at each layer

Task #1 (0.33 pts.) - HTML Form/Validation

Progress: 100%

Details:

- System should allow the user to correct the error without wiping/clearing the form (for the PHP side this includes sticky-form logic to keep the username/email address entries)

Part 1:

Progress: 100%

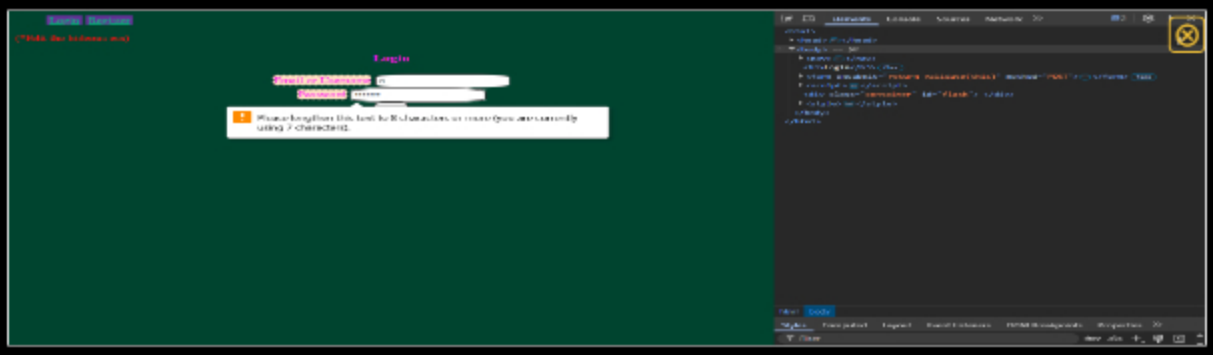
Details:

- Show the code related to the login page's form (HTML validation)
- Show examples of each validation message (you may be able to capture this in one or few screenshots)

```
<h3>Login</h3>
<!   clr26 07/10/2025   >
<!-- When the form is submitted, it uses the validate function from the js script to verify the login information -->
<form onsubmit="return validate(this)" method="POST">
    <div>
        <label for="email">Email or Username</label>
        <input id="email" type="text" name="email" required />
    </div>
    <div>
```

```
    <input type="password" id="pw" name="password" required minlength="8" />
  </div>
  <input type="submit" value="Login" />
</form>
```

HTML ocde/comment



HTML output

Saved: 7/10/2025 9:56:57 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The code calls the javascript validate function which verifies that the information is correct in order to let the user login.

Saved: 7/10/2025 9:56:57 PM

Task #2 (0.33 pts.) - JS Validation (validate() function)

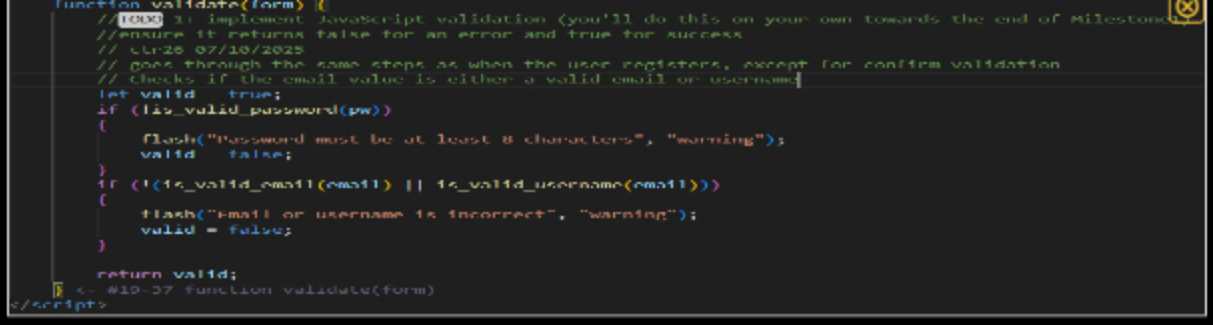
Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code related to the login page's validate() function
- Show examples of each validation message [username format, email format, password format] (you may be able to capture this in one or few screenshots)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag



```
function validate(form) {
    //TODO 1: Implement JavaScript validation (you'll do this on your own towards the end of Milestone 2)
    //ensure it returns false for an error and true for success
    // Ctrl+K 07/10/2025
    // goes through the same steps as when the user registers, except for confirm validation
    // checks if the email value is either a valid email or username
    let valid = true;
    if (!is_valid_password(pw))
    {
        flash("Password must be at least 8 characters", "warning");
        valid = false;
    }
    if (!(is_valid_email(email) || is_valid_username(email)))
    {
        flash("Email or username is incorrect", "warning");
        valid = false;
    }

    return valid;
} <- #10-37 function validate(form)
</script>
```

JS code/comment

JS output

Saved: 7/10/2025 10:05:11 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The code checks for email or username rather than both. It also still validates the password to ensure that the login is correct.

Saved: 7/10/2025 10:05:11 PM

Task #3 (0.33 pts.) - PHP Validation (steps before the DB call)

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code related to the login page's PHP validation

- Show examples of each validation message [username format, email format, password format, invalid password] (you may be able to capture this in one or few screenshots)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag and temporarily override the validate() function or use the provided helper script in the instructions
- Include the outputs related to user doesn't exist and php-side password mismatch

PHP code/comment 1/3

```php
// ctr26 07/10/2025
// Checks the password to ensure it is a valid password before running login data
if (empty($password)) {
    flash("Password must not be empty.", "danger");
    $hasError = true;
}

if (!is_valid_password($password)) {
    //echo "Password too short<br>";
    flash("Password must be at least 8 characters long.", "danger");
    $hasError = true;
} <- #77-81 if (!is_valid_password($password))
```

PHP code/comment 2/3

PHP code/comment 3/3

PHP output

Saved: 7/10/2025 10:12:53 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The code verifies that alal the information is valid before running it through the login functions. The information is compared to the database to check if the user exists. If the information matches, the user is logged in.

Saved: 7/10/2025 10:12:53 PM

Task #2 (0.67 pts.) - Related URLs

Progress: 100%

Details:

- Include the direct link to this file from the Milestone branch (should end in .php)
- Include the heroku prod link to this file (Just grab the base prod url and manually enter the path to the file)
- Include the related pull request link for this feature

URL #1
https://github.com/ColinRafferty7/ctr26-IT202-450/Milestone1/public_html/project/login.php
URL https://github.com/ColinRaff

URL #2
https://ctr26-it202-450-prod-2ae7f60cad14.herokuapp.com/project/login.php
URL https://ctr26-it202-450-prod-2

URL #3
https://github.com/ColinRafferty7/ctr26-IT202-450/pull/20
URL https://github.com/ColinRaff

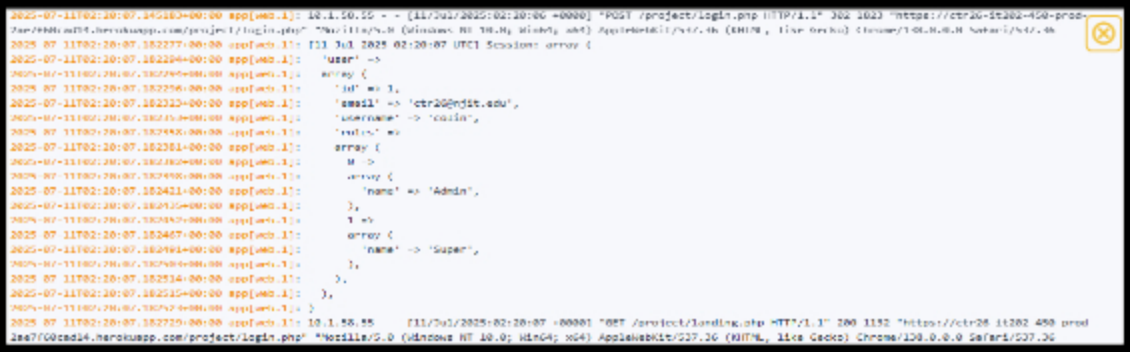
Saved: 7/10/2025 10:17:59 PM

Task #3 (0.67 pts.) - User Session Evidence

Progress: 100%

Details:

- From the heroku logs (Dashboard -> More button -> view Logs), capt
- It should show the id, username, email, and roles



Heroku logs

Saved: 7/10/2025 10:24:42 PM

Section #3: (1 pt.) Feature: User Will Be Able To Logout

Progress: 100%

Task #1 (1 pt.) - Evidence

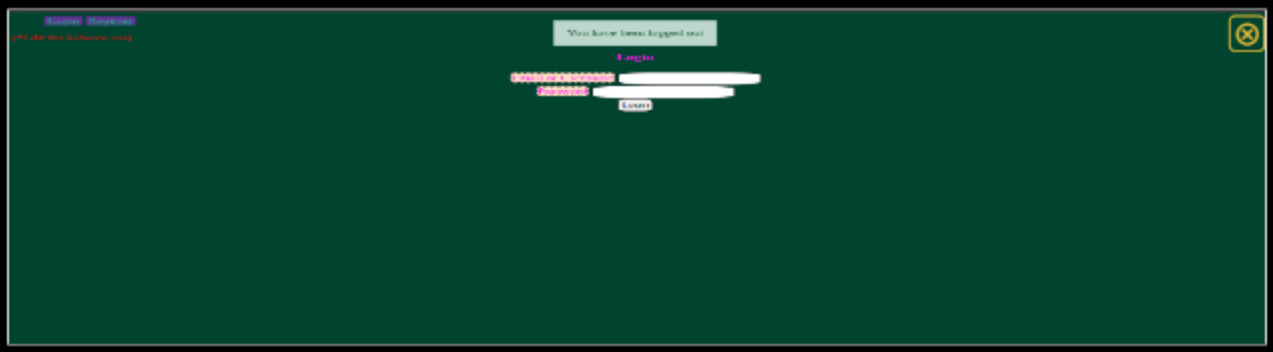
Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the successful logout message
- Show the code snippet of the logout page



Successful logout

```php
<?php
// ctr26 07/10/2025
// Clears all data from the current session in order to logout
// require functions.php to pull in flash()
require(__DIR__ . "/../../lib/functions.php");
reset_session(); // clear session data and start a new session
flash("You have been logged out","success");
header("Location: $BASE_PATH/login.php"); // redirect back to login
```

Logout code/comment

Saved: 7/10/2025 10:33:29 PM

Part 2:

Progress: 100%

Details:

- Include the pull request url for this feature

URL #1
https://github.com/ColinRafferty7/ctr26-
IT202-450

URL
https://github.com/ColinRafferty7

Saved: 7/10/2025 10:33:29 PM

Part 3:

Progress: 100%

Details:

- Briefly explain how the logout process works and how the user is officially logged off

Your Response:

The user is logged off by the session data being reset, and the user being redirected to the login page. This ensures than any previous interactions on the page are wiped, and that the user is not on any page where a login is required.

Saved: 7/10/2025 10:33:29 PM

Section #4: (2 pts.) Feature: Security Rules

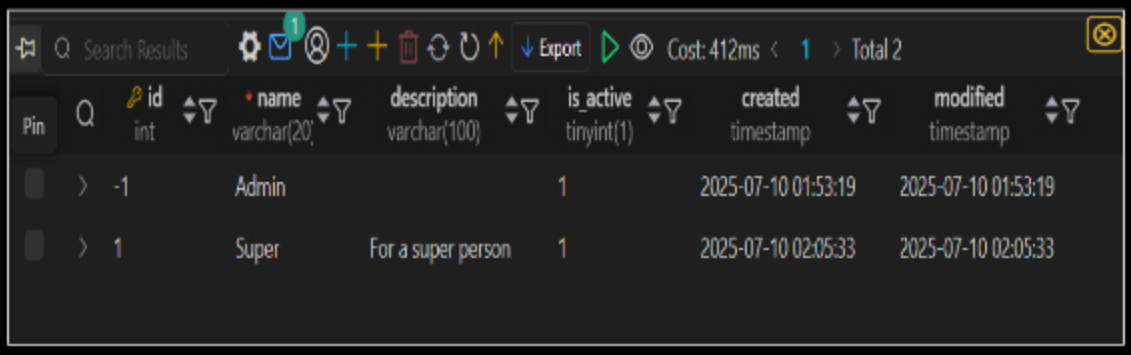
Progress: 100%

Task #1 (1 pt.) - Database Tables

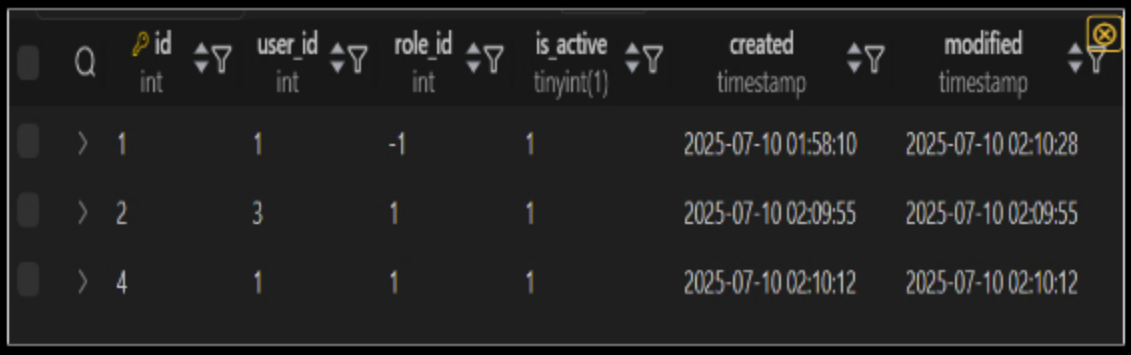
Progress: 100%

Details:

- From the vs code mysql tool, show both the Roles and UserRoles tal



Roles table



UserRoles table

Saved: 7/10/2025 10:34:25 PM

Task #2 (1 pt.) - Demo Security Rules

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the message related to needing to be logged in (i.e., try to manuall
- Show the message related to not having the proper permission/role (i.e.,
- Include a snippet of the login check function
- Include a snippet of the role check function

Login required

No permission

Saved: 7/10/2025 10:42:11 PM

Part 2:

Progress: 100%

Details:

- Include the pull request url for this feature

URL #1
https://github.com/ColinRafferty7/ctr26-IT202-450

URL
https://github.com/ColinRafferty7

Saved: 7/10/2025 10:42:11 PM

Part 3:

Progress: 100%

Details:

```
- Briefly explain how the login check code works
- Briefly explain how the role check code works
```

Your Response:

The login check works by using the user_helpers function, is_logged_in, which outputs the users logged in status. If the user is not logged in, they are redirected to the login page. The role check

works by using the user_helpers function, check_role, to see if the currents sessions user has the admin role. If not, they are redirected to the landing page.

Saved: 7/10/2025 10:42:11 PM

Section #5: (2 pts.) Feature: User Profile

Progress: 83%

Task #1 (1 pt.) - Validation

Progress: 66%

Details:

- Show the relevant code snippets for each validation layer
- Show the relevant demo of each validation layer
- Briefly explain how the validation steps work at each layer

Task #1 (0.33 pts.) - HTML Form/Validation

Progress: 100%

Details:

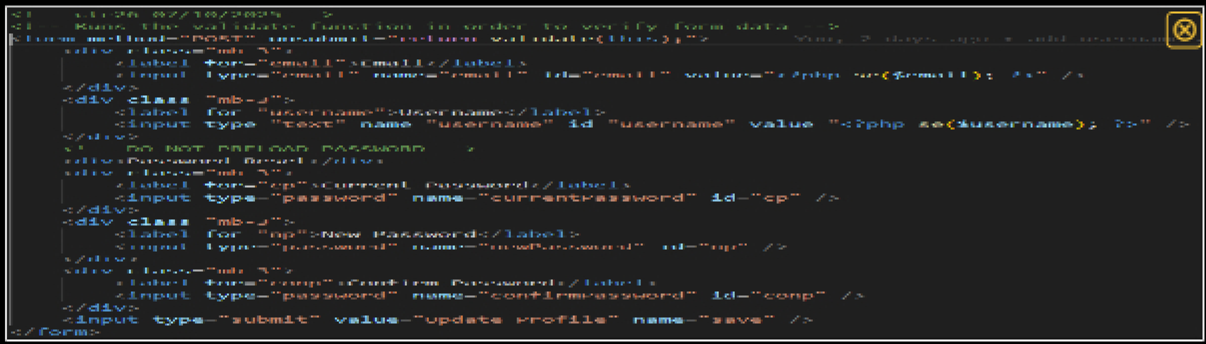
- System should allow the user to correct the error without wiping/clearing the form (for the PHP side this includes sticky-form logic to keep the username/email address entries)

Part 1:

Progress: 100%

Details:

- Show the code related to the profile page's form (HTML validation)
- Show examples of each validation message (you may be able to capture this in one or few screenshots)



HTML code/comment

HTML output

Saved: 7/10/2025 10:56:40 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The code validates the password by calling the validate function with the form data. The script then verifies that all the data is valid and allows the form to be submitted.

Saved: 7/10/2025 10:56:40 PM

Task #2 (0.33 pts.) - JS Validation (validate() function)

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code related to the profile page's validate() function
- Show examples of each validation message [username format, email format, password format, password doesn't match confirm] (you may be able to capture this in one or few screenshots)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag

JS code/comment

JS output

Saved: 7/10/2025 10:57:44 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The code works by first checking if the newly entered password is a valid password. Then, the entered password is compared to the confirm to ensure that they match. If they match, the new password is set for the users profile.

Saved: 7/10/2025 10:57:44 PM

Task #3 (0.33 pts.) - PHP Validation (steps before the DB call)

Progress: 0%

Part 1:

Progress: 0%

Details:

- Show the code related to the profile page's PHP validation
- Show examples of each validation message [username format, email format, password format, invalid password, password doesn't match confirm, username

taken, email taken] (you may be able to capture this in one or few screenshots)

- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag and temporarily override the validate() function or use the provided helper script in the instructions

Missing Caption

Saved: 7/5/2025 3:22:23 PM

Part 2:

Progress: 0%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

Missing Response

Saved: 7/5/2025 3:22:23 PM

Task #2 (1 pt.) - Related URLs

Progress: 100%

Details:

- Include the direct link to this file from the Milestone branch (should end in .php)
- Include the heroku prod link to this file (Just grab the base prod url and manually enter the path to the file)
- Include the related pull request link for this feature

URL #1

https://github.com/ColinRafferty7/ctr26-IT202-450-Milestone1/public_html/project/profile.php

URL

https://github.com/ColinRaff

URL #2

URL

https://ctr26-it202-450-prod-2ae7f60cad14.herokuapp.com/project/profile.php

URL #3
https://github.com/ColinRafferty7/ctr26-IT202-450/pull/14

URL
https://github.com/ColinRaff

Saved: 7/10/2025 11:03:48 PM

Section #6: (1 pt.) Misc

Progress: 100%

Task #1 (0.33 pts.) - Github Details

Progress: 100%

Part 1:

Progress: 100%

Details:
From the Commits tab of the Pull Request screenshot the commit history

Commit history

Saved: 7/10/2025 11:05:05 PM

Part 2:

Progress: 100%

Details:
Include the link to the Pull Request (should end in `/pull/#`)

URL #1
https://github.com/ColinRafferty7/ctr26-IT202-450/pull/8

URL
https://github.com/ColinRafferty7

Saved: 7/10/2025 11:05:05 PM

Task #2 (0.33 pts.) - WakaTime - Activity

Task #2 (0.33 pts.) - WakaTime - Activity

Progress: 100%

Details:

- Visit the WakaTime.com Dashboard
- Click `Projects` and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary

Heroku bottom

Heroku top

Saved: 7/10/2025 11:07:24 PM

Task #3 (0.33 pts.) - Reflection

Progress: 100%

Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

In this assignment I learned how website handle user logins and accounts. Going through each step and seeing how lengthy the process actually is for something as simple as logging in showed me how much work actually goes into creating a smooth experience for the user.

Saved: 7/10/2025 11:09:04 PM

Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

The easiest part of the assignment was going through all of the lessons and connecting all of the code together. Seeing every part click together nicely was pretty satisfying and I was able to understand it step by step.

Saved: 7/10/2025 11:10:03 PM

Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

The hardest part of this assignment was understanding what changes I was supposed to make for many of the steps. The instructions were still a bit confusing even after watching all of the video guides.

Saved: 7/10/2025 11:11:11 PM